

LLDD BE - API Document List and Search

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจสอบ และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	BE
Estimate	26 ชั่วโมง = implementation 20 + unit test 6 (30%)
Owner	Butsaba <But> Podamrong
Target repository	SBP/srm-sps-spsap-store-backend (NestJS + TypeORM · schema sps_store) + SBP/srm-sps-spsap-sbp-bff (forward ผ่าน client service · ไม่มี DB) สำหรับเส้นที่ FE เรียก
Objective	ออกแบบ APIs สำหรับงานรอดำเนินการและค้นหาเอกสารที่เกี่ยวข้อง

Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

1.1 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
ถูกเรียกจาก	LLDD-FE-Document-Lists.pdf	GET /api/v1/sgi/document · GET /api/v1/sgi/document/tasks
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope {success, data} · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
workflow engine	LLDD-BE-Workflow-Engine-Definition.pdf	นิยาม state/route/event ที่หัวข้อ Workflow Trigger Event Contract เรียกใช้
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (x-api-key / x-user-*) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Database-Structure.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Screen / Functional Scope

- Inbox tasks API
- Document search API
- Pagination
- Status/year filter
- Abnormal row support

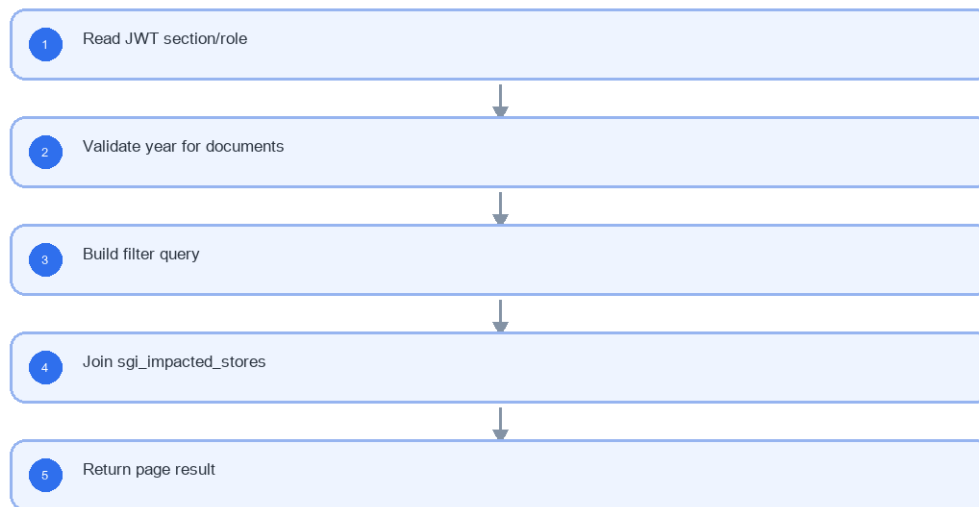
3. Screenshot Reference

ไม่มีภาพหน้าจอสำหรับหัวข้อนี้ — เป็นเอกสารฝั่ง Backend/Batch ที่ไม่มี UI (ภาพหน้าจอทั้งหมดอยู่ในเอกสารชุด FE)

4. Implementation Flow & Sequence Diagram (Reference)

4.1 Implementation Flow (ลำดับชั้นการทำงาน)

LLDD BE - API Document List and Search

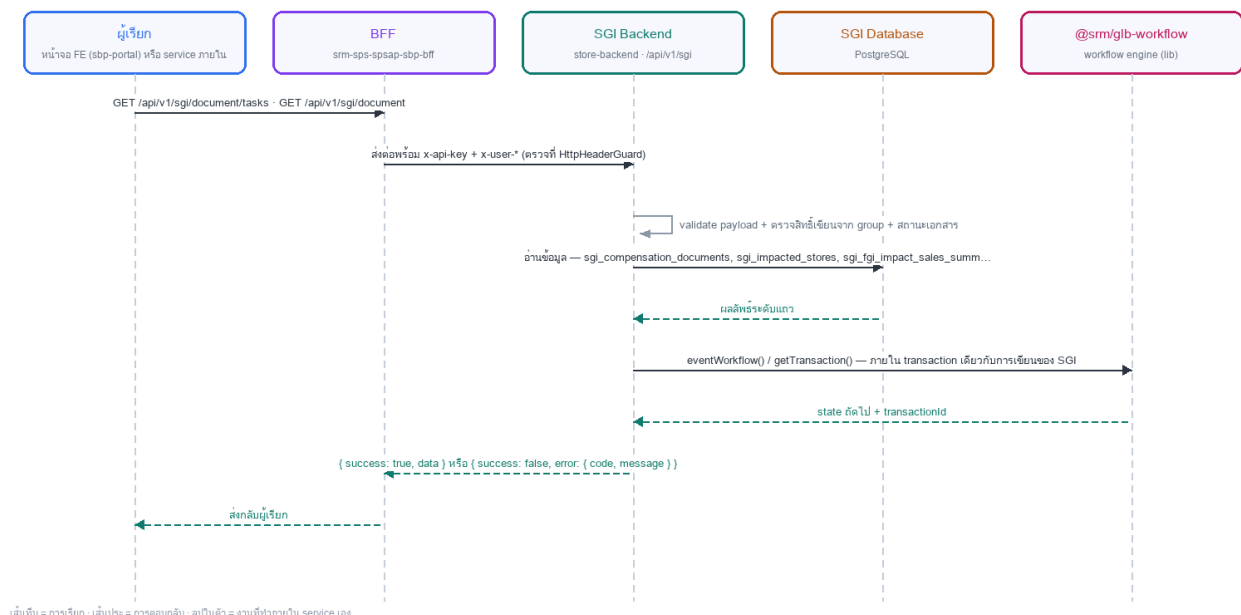


รูปที่ 1: Implementation flow reference: LLDD BE - API Document List and Search

4.2 Sequence Diagram (ใครคุยกับใคร ลำดับไหน)

ผู้แสดงและลำดับข้อความในภาพนี้สร้างจาก endpoint ในหัวข้อ 7 และตารางในหัวข้อ Reference DB Mapping ของเอกสารฉบับนี้เอง จึงตรงกับสัญญาณเสมอ

Sequence — LLDD BE - API Document List and Search



รูปที่ 2: Sequence diagram: LLDD BE - API Document List and Search

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
docNo	YYYY/xxxx	required when opening existing document	ใช้ปี ค.ศ. และ running 5 หลัก (มติ 2026-08-06)
storeCode	string 5 digits	numeric length = 5	แสดง leading zero
amount	number, 2 decimals	>= 0	format #, ##0.00 บาท
percent	number, 2 decimals	0-100	ใช้ % และรวม allocation ต้องเท่ากับ 100 — B5: เพิ่ม/ลบวันที่กระทบเพิ่มเมื่อไร ต้องเกลี่ยใหม่ทั้งชุดแล้วคำนวณ compensateAmount ของทุกแถวใหม่ ไม่ใช่เฉพาะแถวที่เพิ่ม
sourceSystem	enum	ALLMAP / USER	B5 ที่มาของแถวร้านเปิดใหม่ — ALLMAP ระบบ default ให้อัตโนมัติ (Job 9) · USER เจ้าหน้าที่ SBP DSA คีย์เองจากเอกสารแจ้งของหน่วยงานส่งเสริม (ฝั่ง To-Be · SDD สไลด์ 7) · ซ้ำ (doc_no, new_store_code) ให้คืน 409
date	DD/MM/YYYY	valid date	payload เป็น ISO ค.ศ. · FE แสดง ค.ศ. เป็นค่าเริ่มต้น (DatePicker buddhistEra=false) แสดง พ.ศ. เฉพาะจุดที่เปิด flag
attachment	file	<= 5 MB	รองรับ vsd, dwg, afp, pdf, mda, zip, wav, mp3, gif, jpg, tif, tiff, htm, html, txt, xml, mpg, mov, ivs, doc, docx, xls, xlsx, pps, ppt, pot, csv
year	ค.ศ. YYYY	required for /sgi/document	ไม่ระบุคืน 400 ตามข้อกำหนดทางธุรกิจ · BE ผ่าน toAD() เพื่อ client ส่ง พ.ศ.
page/size	integer	page>=1 size<=100	pagination

5.9 Input / Progress / Output Contract

Stage	Contract for implementation
Input	GET /api/v1/sgi/document/tasks; GET /api/v1/sgi/document
Progress	Read JWT section/role; Validate year for documents; Build filter query; Join sgi_impacted_stores
Output	ไม่มีตารางที่เอกสารนี้เขียนเอง — output คือ response ตาม envelope กลาง {success, data} และรอรอยที่ตรวจย้อนได้ (log / sgi_consideration_logs / workflow_history ของ engine)

5.90 Endpoint Implementation Contract

Endpoint	Use-case owner	Service/repository behavior	Definition of done
GET /api/v1/sgi/document/tasks	Inbox tasks API	Read JWT section/role	year missing fails for /sgi/document
GET /api/v1/sgi/document	Document search API	Validate year for documents	leading zero storeCode preserved

5.91 Backend Execution Sequence

Step	Behavior specific to this LLDD	Failure/test evidence
1	Read JWT section/role	tasks by section
2	Validate year for documents	documents missing year
3	Build filter query	store search
4	Join sgi_impacted_stores	empty result
5	Return page result	— (ยังไม่มี test เฉพาะขั้นนี้ · ครอบด้วย test รวมของเอกสารในหัวข้อ 11)

5.92 Workflow Trigger Event Contract

งานขั้นนี้ ต้องเรียก workflow engine ตามตารางด้านล่าง · ชื่อ function ยึด API 8 ตัวของ @srm/glb-workflow ตามขีด Detail ของ **TSM SRM LLDD SBP workflow ไฟล์ 1.2** — รายละเอียด signature และตารางที่ engine เขียน ดู

LLDD-BE-Workflow-Engine-Definition.pdf หัวข้อ 5.3

จุดที่เรียก (call site)	Engine function	พารามิเตอร์หลัก	กติกา / transaction boundary
กล่องงานรอดำเนินการ	getPendingFlowByUser	userData, versionId	เป็นแหล่งความจริงของรายการรอดำเนินการ · section 06 ต้อง union เอกสารที่จบด้วย หยุดชัดเจน (stoppedReopenable) เอง

- กติกาหลัก: ตาราง sps_store.workflow_* (13 ตาราง) เป็นของ lib — SGI R เท่านั้น ห้าม INSERT/UPDATE/DELETE ตรงในทุกกรณี
- ทุกการเรียก engine ต้องผ่านตัวห่อกลาง WorkflowGateway ที่นิยามใน LLDD-BE-API-Common-Contracts.pdf (timeout · retry · map error เข้า envelope) ห้าม import lib ตรงจาก service
- unit test ต้อง mock engine และครอบอย่างน้อย: เรียกสำเร็จ · engine โยน error แล้ว rollback ฝั่ง SGI ครอบ · เรียกซ้ำด้วย referenceld เดิมไม่เกิดผลซ้ำ

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
Inbox tasks	GET	task.service.searchOpenTasks	return waiting list
Document search	GET	document.service.search	return related list

7. API Contract

GET /api/v1/sgi/document/tasks

Inbox tasks API

Query Params
<pre>{ "sectionCode": "06", "page": 1, "size": 20 }</pre>

Request Field Schema

Field	Type	Required	Constraint / Meaning
sectionCode	string	No	canonical code; do not replace with display label

Field	Type	Required	Constraint / Meaning
page	integer	No	>= 1; default 1
size	integer	No	1..100; default 20

Response

```
{
  "items": [
    {
      "docNo": "2026/00123",
      "waitingDays": 3
    }
  ]
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
items	array<object>	Yes	JSON array; element type shown in Type column
items[].docNo	string	Yes	ค.ศ. YYYY/xxxxx
items[].waitingDays	integer	Yes	UTF-8; use value domain described by endpoint purpose

GET /api/v1/sgi/document

Document search API

Query Params

```
{
  "year": 2026,
  "storeCode": "00788",
  "status": "06",
  "page": 1
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
year	integer	Yes	UTF-8; use value domain described by endpoint purpose
storeCode	string	No	exactly 5 digits; preserve leading zero
status	string	No	UTF-8; use value domain described by endpoint purpose
page	integer	No	>= 1; default 1

Response

```
{
  "items": [
    {
      "docNo": "2026/00123",
      "statusCode": "06"
    }
  ]
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
items	array<object>	Yes	JSON array; element type shown in Type column
items[].docNo	string	Yes	ค.ศ. YYYY/xxxxx
items[].statusCode	string	Yes	canonical code; do not replace with display label

8. Reference DB Mapping (No Database Page Work)

ส่วนนี้เป็นข้อมูลอ้างอิงสำหรับการ implement API/Job เท่านั้น ไม่ใช้งานสร้างหน้า Database, ไม่ใช้งานออกแบบ DB page และไม่ถูกนับเป็น deliverable แยกของ FE/BE

Table / Object	R/W	Usage
workflow_transaction / workflow_approver (@srm/glb-workflow)	R	อ่าน inbox ผ่าน getPendingFlowByUser() · เฉพาะ section 06 ต้อง union เอกสารที่จบด้วยหยุดขัดเซย์ประกันรายได้ เข้ามาด้วย (stoppedReopenable)
sgi_compensation_documents	R	ค้นเอกสารตาม year/status/store
sgi_impacted_stores	R	ชื่อร้าน ภาค และข้อมูลร้าน
sgi_fgi_impact_sales_summaries	R	flag ข้อมูลผิดปกติ/ยอดขายไม่ครบ 60 วัน
sgi_consideration_logs	R	ผลการพิจารณาสุดท้าย — คัดเอกสารที่จบด้วยหยุดขัดเซย์ประกันรายได้ เข้าคิวของ section 06 (SDD สไลด์ 46 ข้อ 1.9)

9. Skeleton Code (store-backend + BFF)

โครงโค้ดตั้งต้นของเอกสารฉบับนี้ ยึด convention จริงของ srm-sps-spsap-store-backend (NestJS 11 + TypeORM, schema sps_store, custom provider DATA_SOURCE ที่ route SELECT ไป slave pool) และ srm-sps-spsap-sbp-bff (ไม่มี DB, forward ผ่าน client service). ทุกจุดที่ต้องเติมกำกับด้วย // TODO: และ response ทุกเส้นถูกห่อเป็น {success, data} โดย ResponseInterceptor อยู่แล้ว จึงห้าม service ห่อซ้ำ

9.1 ไฟล์ที่ต้องสร้าง

Path	หน้าที่
store-backend · src/modules/sgi-document-list-search/sgi-document-list-search.controller.ts	route ทั้งหมดของเอกสารนี้ (2 เส้น) + @UseGuards(HttpHeaderGuard) + @UserId()
store-backend · src/modules/sgi-document-list-search/sgi-document-list-search.service.ts	business logic — inject 'DATA_SOURCE' แล้วยิง raw SQL, mutation ใช้ QueryRunner transaction
store-backend · src/modules/sgi-document-list-search/sgi-document-list-search.sql.ts	เก็บ SQL ต่อ endpoint (คัดจากหัวข้อ 10) แยกออกจาก service ให้ทดสอบ/รีวิวง่าย
store-backend · src/modules/sgi-document-list-search/dto/sgi-document-list-search.dto.ts	DTO + class-validator ตาม validation ในหัวข้อฟิลด์ของเอกสารนี้
store-backend · src/modules/sgi-document-list-search/sgi-document-list-search.module.ts	ประกอบ controller/service/providers แล้ว register ที่ app.module.ts
store-backend · src/entities/sgi-compensation-documents.entity.ts	entity ของ sgi_compensation_documents (@Entity({schema: process.env.DB_SCHEMA}), ไม่ประกาศ relation) — entity รวมหลายเอกสาร: ประกาศครั้งเดียวแล้วอ้างอิง อย่าสร้างซ้ำ
store-backend · src/entities/sgi-impacted-stores.entity.ts	entity ของ sgi_impacted_stores (@Entity({schema: process.env.DB_SCHEMA}), ไม่ประกาศ relation)
store-backend · src/entities/sgi-fgi-impact-sales-summaries.entity.ts	entity ของ sgi_fgi_impact_sales_summaries (@Entity({schema: process.env.DB_SCHEMA}), ไม่ประกาศ relation)

Path	หน้าที่
store-backend · src/providers/sgi/sgi.ts	repository provider แบบ factory ผูก token string กับ DATA_SOURCE — ไฟล์รวมของทุกเอกสาร BE ให้ merge array เพิ่ม ห้ามเขียนทับ
store-backend · sql/deploy-sgi-document-list-search.sql	DDL production แบบ idempotent (ที่นี่ไม่ใช่ migration เป็นหลัก)
BFF · src/common/client-services/sgi-client.service.ts	client ต่อจาก BaseClientService ตั้ง baseUrl + x-api-key ตอน onModuleInit
BFF · src/modules/sgi-document-list-search/sgi-document-list-search .controller.ts	route ฟังก์ BFF prefix /bff/sgi/... + @UseGuards(AuthGuard('jwt'))
BFF · src/modules/sgi-document-list-search/sgi-document-list-search .service.ts	แนบ x-user-id / x-user-group-id / x-user-permissions แล้ว forward ไป backend

9.2 Controller (store-backend)

```
// src/modules/sgi-document-list-search/sgi-document-list-search.controller.ts
import { Controller, Get, Query, UseGuards } from '@nestjs/common';
import { HttpHeaderGuard } from '../../guards/http-header.guard';
import { UserId } from '../../common/decorators/user-id.decorator';
import { SgiDocumentListSearchService } from './sgi-document-list-search.service';
import { DocumentListSearchQueryDto } from './dto/sgi-document-list-search.dto';

// LLDD BE - API Document List and Search
// BFF เรียกด้วย x-api-key และแนบ x-user-id / x-user-group-id / x-user-permissions มาให้
@Controller('sgi/sgi/document')
@UseGuards(HttpHeaderGuard)
export class SgiDocumentListSearchController {
  constructor(private readonly service: SgiDocumentListSearchService) {}

  // GET /api/v1/sgi/document/tasks — Inbox tasks API
  @Get('document/tasks')
  getSgiDocumentTasks(@Query() query: DocumentListSearchQueryDto, @UserId() userId: string) {
    // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.getSgiDocumentTasks(query, userId);
  }

  // GET /api/v1/sgi/document — Document search API
  @Get('document')
  getSgiDocument(@Query() query: DocumentListSearchQueryDto, @UserId() userId: string) {
    // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.getSgiDocument(query, userId);
  }
}
```

9.3 DTO + Validation

```
// src/modules/sgi-document-list-search/dto/sgi-document-list-search.dto.ts
import { Type } from 'class-transformer';
import {
  isArray, isBoolean, isIn, isInt, isEmpty, isNumber, isObject, isOptional,
  isString, matches, max, maxLength, min,
} from 'class-validator';

// ValidationPipe ระดับ global ตั้ง whitelist + forbidNonWhitelisted + transform ไว้แล้ว (main.ts)
// property ที่ไม่ประกาศที่นี่จะถูก reject เป็น 400 อัตโนมัติ

// query รวมของ GET ทุกเส้นในโมดูลนี้ (path param ไข่ @Param แยก)
export class DocumentListSearchQueryDto {
  /** required เฉพาะบาง endpoint — ตรวจสอบซ้ำใน service */
  @IsOptional()
  @IsString()
  sectionCode?: string;

  /** pagination */
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  page?: number;

  /** pagination */
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  @Max(100)
  size?: number;
}
```



```

/** ไม่ระบุคืน 400 ตาม ข้อกำหนดทางธุรกิจ · BE ผ่าน toAD() เพื่อ client ส่ง พ.ศ. · required เฉพาะบาง endpoint... */
@IsOptional()
@Type(() => Number)
@IsInt()
year?: number;

/** แสดง leading zero · required เฉพาะบาง endpoint — ตรวจสอบใน service */
@IsOptional()
@IsString()
@Matches(/^\d{5}$/, { message: 'รหัสร้านต้องเป็นตัวเลข 5 หลัก และคงเลขศูนย์นำหน้า' })
storeCode?: string;

/** required เฉพาะบาง endpoint — ตรวจสอบใน service */
@IsOptional()
@IsString()
status?: string;
}

```

9.4 Service (inject `DATA\_SOURCE` + raw SQL)

service ประกาศ method ครบทุกเส้นที่ controller เรียก และ signature มาจากแหล่งเดียวกับ controller (จำนวน/ลำดับพารามิเตอร์จึงตรงกันเสมอ) — เส้นที่ยังไม่ได้ implement เป็น stub ที่ throw new `NotImplementedException(...)` ให้ TypeScript compile ผ่านตั้งแต่วันแรก

```

// src/modules/sgi-document-list-search/sgi-document-list-search.service.ts
import { Inject, Injectable, Logger, NotFoundException, NotImplementedException } from '@nestjs/common';
import { DataSource } from 'typeorm';
import { WorkflowService } from '../workflow/workflow.service';
import { SGI_SQL } from './sgi-document-list-search.sql';

@Injectable()
export class SgiDocumentListSearchService {
  private readonly logger = new Logger(SgiDocumentListSearchService.name);
  // versionId ของ workflow ประกันรายได้ (ตั้งใน env เหมือน COOPERATION_WORKFLOW_VERSION_ID)
  private readonly versionId = Number(process.env.SGI_WORKFLOW_VERSION_ID);

  constructor(
    // DATA_SOURCE override query(): SELECT/WITH ไป slave pool, write ไป master
    @Inject('DATA_SOURCE') private readonly dataSource: DataSource,
    private readonly workflow: WorkflowService,
  ) {}

  // GET /api/v1/sgi/document/tasks — Inbox tasks API
  async getSgiDocumentTasks(query: DocumentListSearchQueryDto, userId: string) {
    const page = Number(query.page ?? 1);
    const size = Math.min(Number(query.size ?? 20), 100);
    // SQL เต็มอยู่ในหัวข้อ Database SQL ของเอกสารนี้ (คือ 'GET /api/v1/sgi/document/tasks')
    // □ □ SQL ตัวอย่างบางเส้นเขียนด้วย named parameter (:size/:offset) แต่ dataSource.query()
    // รับเฉพาะ positional $1..$n — ต้องแปลงชื่อเป็นลำดับก่อน หรือใช้ QueryBuilder แทน
    const rows = await this.dataSource.query(SGI_SQL.getSgiDocumentTasks, [
      // TODO: เรียงพารามิเตอร์ให้ตรงกับ $1..$n ของ SQL จริง
      userId, (page - 1) * size, size,
    ]);
    // TODO: total ต้องมาจาก COUNT(*) แยก query หรือ window function ไม่ใช่ rows.length
    return { page, size, total: rows.length, items: rows };
  }

  // GET /api/v1/sgi/document — Document search API
  async getSgiDocument(query: DocumentListSearchQueryDto, userId: string) {
    // TODO: implement ตาม business rule ของ GET /api/v1/sgi/document
    // (SQL อยู่ในหัวข้อ Database SQL คือ 'GET /api/v1/sgi/document')
    throw new NotImplementedException('getSgiDocument ยังไม่ implement');
  }
}

```

9.5 Workflow (`@srm/glb-workflow`)

□ ชื่อ function ของ engine — ยึด LLDD ของ lib (ยืนยันแล้ว 2026-08-14) · API จริงคือ 8 ตัวตามซีต Detail ของ SBP/TSM-SRM-LLDD SBP workflow 1.2.xlsx (เอกสารของ lib เอง): `initializeWorkflow` · `eventWorkflow` · `getPermissionEvents` · `getHistory` · `getTransaction` · `getPendingFlowByUser` · `getWorkflowsByUser` · `addPreApprover` · ชื่อที่เคยขัดกันไม่ใช่ชื่อ API — *Trigger Event* เป็นชื่อหัวข้อขั้นตอนภายใน `eventWorkflow` และ *UseCase* เป็น class ที่ store-backend ห่อไว้ใช้เอง (ดู LLDD-BE-Workflow-Engine-Definition.pdf หัวข้อ 5.3)

Endpoint	Use case ที่ต้องเรียก	เหตุผล
GET /api/v1/sgi/document/tasks	<code>getPendingFlowByUser()</code>	inbox งานค้างของ userId/groupId ที่ BFF ส่งมาใน header

```
// src/modules/sgi-document-list-search/sgi-document-list-search.workflow.ts (หรือรวมไว้ใน service เดียวกัน)
// WorkflowService = wrapper ของ @srm/glb-workflow ที่ store-backend มีอยู่แล้ว
// (DataSource แยกชื่อ 'workflow-connection', ทุก use case ห่อด้วย TypeOrmUnitOfWork)

// inbox งานค้าง — ใช้ร่วมกับ /api/workflow/pending ของ backlog เดิมได้
const pending = await this.workflow.getPendingFlowByUser({
  userData: { userId: Number(userId), groupId: Number(groupId) },
  versionId: this.versionId,
});
// TODO: join referenceId (= doc_no) กลับไปที่ sgi_compensation_documents เพื่อเติมข้อมูลเอกสาร
```

9.6 Entity (TypeORM)

```
// src/entities/sgi-compensation-documents.entity.ts
import { Column, Entity, PrimaryColumn } from 'typeorm';

@Entity({ name: 'sgi_compensation_documents', schema: process.env.DB_SCHEMA })
export class CompensationDocument {
  @PrimaryColumn({ name: 'doc_no', type: 'varchar', length: 12 })
  docNo: string;

  @Column({ name: 'impact_process_id', type: 'bigint', nullable: true })
  impactProcessId?: number;

  @Column({ name: 'impacted_store_code', type: 'char', length: 5 })
  impactedStoreCode: string;

  @Column({ name: 'status_code', type: 'varchar', length: 2 })
  statusCode: string;

  @Column({ name: 'current_section_code', type: 'varchar', length: 2 })
  currentSectionCode: string;

  @Column({ name: 'round_no', type: 'int', nullable: true })
  roundNo?: number;

  @Column({ name: 'loop_no', type: 'int', nullable: true })
  loopNo?: number;

  @Column({ name: 'statement_id', type: 'varchar', length: 30, nullable: true })
  statementId?: string;

  @Column({ name: 'statement_date', type: 'date', nullable: true })
  statementDate?: Date;

  @Column({ name: 'account_year', type: 'int', nullable: true })
  accountYear?: number;

  @Column({ name: 'account_month', type: 'int', nullable: true })
  accountMonth?: number;

  @Column({ name: 'compensate_amount', type: 'numeric', precision: 15, scale: 2, nullable: true })
  compensateAmount?: string;

  @Column({ name: 'allmap_url', type: 'text', nullable: true })
  allmapUrl?: string;

  @Column({ name: 'approver_snapshot', type: 'jsonb', nullable: true })
  approverSnapshot?: Record<string, unknown>;

  @Column({ name: 'created_at', type: 'timestampz', nullable: true })
  createdAt?: Date;

  @Column({ name: 'updated_at', type: 'timestampz', nullable: true })
  updatedAt?: Date;

  // TODO: ตรวจสอบความยาว/precision กับ DDL จริงใน sql/deploy-sgi-*.sql ก่อน merge
  // entity ชุดนี้ไม่ประกาศ relation ตาม convention (join ด้วย raw SQL)
}

// src/entities/sgi-impacted-stores.entity.ts
import { Column, Entity, PrimaryColumn } from 'typeorm';

@Entity({ name: 'sgi_impacted_stores', schema: process.env.DB_SCHEMA })
export class ImpactedStore {
  @PrimaryColumn({ name: 'store_code', type: 'char', length: 5 })
  storeCode: string;

  @Column({ name: 'store_name', type: 'varchar', length: 200 })
  storeName: string;

  @Column({ name: 'zone_code', type: 'varchar', length: 10, nullable: true })
  zoneCode?: string;
}
```

```

@Column({ name: 'region_code', type: 'varchar', length: 10, nullable: true })
regionCode?: string;

@Column({ name: 'store_type', type: 'varchar', length: 5, nullable: true })
storeType?: string;

@Column({ name: 'transfer_sbp_date', type: 'date', nullable: true })
transferSbpDate?: Date;

@Column({ name: 'is_active', type: 'boolean', default: true })
isActive: boolean;

// TODO: ตรวจสอบความยาว/precision กับ DDL จริงใน sql/deploy-sgi-*.sql ก่อน merge
// entity ชุดนี้ไม่ประกาศ relation ตาม convention (join ด้วย raw SQL)
}

```

ตารางที่เหลือของเอกสารนี้ (sgi_fgi_impact_sales_summaries, sgi_consideration_logs) ใช้รูปแบบ entity เดียวกัน — คอลัมน์อ้างอิงจาก Database design

ตารางที่ไม่ต้องสร้าง entity เพราะใช้ของระบบเดิม/workflow engine:

Object	R/W	ใช้ของระบบเดิมตัวไหน
workflow_transaction	R	workflow engine @srm/glb-workflow
workflow_approver	R	workflow engine @srm/glb-workflow

9.7 Repository Providers + Module wiring

```

// src/providers/sgi/sgi.ts — repository provider แบบ factory (ไม่ใช่ TypeOrmModule.forFeature)
// convention ของไฟล์ provider providers คือ 1 ไฟล์ต่อโดเมน ตั้งชื่อตามโดเมน (business_user/business_user.ts,
// common_code/common_code.ts ...) ไม่ใช่ index.ts
//
// □ □ ไฟล์นี้ใช้ร่วมกันทุกเอกสาร BE ของ SGI — ให้ **merge array เพิ่ม** เข้าไฟล์เดิม ห้ามเขียนทับ
// (ชื่อ const แยกต่อเอกสารไว้แล้วเพื่อไม่ให้ชนกัน)
import { DataSource } from 'typeorm';
import { CompensationDocument } from '../entities/sgi-compensation-documents.entity';
import { ImpactedStore } from '../entities/sgi-impacted-stores.entity';
import { FgiImpactSalesSummary } from '../entities/sgi-feri-impact-sales-summaries.entity';

export const sgiDocumentListSearchProviders = [
  {
    provide: 'SGI_COMPENSATION_DOCUMENT_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(CompensationDocument),
    inject: ['DATA_SOURCE'],
  },
  {
    provide: 'SGI_IMPACTED_STORE_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(ImpactedStore),
    inject: ['DATA_SOURCE'],
  },
  {
    provide: 'SGI_FGI_IMPACT_SALES_SUMMARIES_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(FgiImpactSalesSummary),
    inject: ['DATA_SOURCE'],
  },
];

// src/modules/sgi-document-list-search/sgi-document-list-search.module.ts
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
import { DatabaseModule } from '../database/database.module';
// UserContextMiddleware อ่าน header x-user-id แล้วเซต request.userId ที่ @UserId() ใช้
// — app.module.ts **ไม่ได้** apply แบบ global (มีแค่ HttpContext/LoggerContext) แต่ละโมดูลต้อง apply เอง
// (ดู evaluation-process.module.ts / inform-evaluate.module.ts / cooperation-request.module.ts)
import { UserContextMiddleware } from '../common/middleware/user-context.middleware';
import { WorkflowModule } from '../workflow/workflow.module';
import { sgiDocumentListSearchProviders } from '../providers/sgi/sgi';
import { SgiDocumentListSearchController } from './sgi-document-list-search.controller';
import { SgiDocumentListSearchService } from './sgi-document-list-search.service';

@Module({
  imports: [DatabaseModule, WorkflowModule],
  controllers: [SgiDocumentListSearchController],
  providers: [SgiDocumentListSearchService, ...sgiDocumentListSearchProviders],
  exports: [SgiDocumentListSearchService],
})
export class SgiDocumentListSearchModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    // ถ้าไม่ apply ตรงนี้ userId จะเป็น undefined เขียน ๆ ทุก endpoint
    consumer.apply(UserContextMiddleware).forRoutes(SgiDocumentListSearchController);
  }
}

```

```

}
}
// TODO: register module นี้ใน app.module.ts (imports) พร้อมกับโมดูล SGI ตัวอื่น

```

9.8 BFF Proxy (module + controller + client service)

BFF ยังไม่มีที่เจอรูปประกันรายได้เลย จึงต้องสร้าง module ใหม่ + client service ใหม่ทั้งชุด และเลือก prefix แบบเดียวทั้งโมดูล (ที่นี้ใช้ `/bff/sgi/...`) เพื่อไม่ให้ปนแบบที่มี/ไม่มี `/bff` เหมือนโมดูลเดิม

```

// src/common/client-services/sgi-client.service.ts
import { Injectable, Logger, OnModuleInit } from '@nestjs/common';
import { BaseClientService } from './base-client.service';

@Injectable()
export class SgiClientService extends BaseClientService implements OnModuleInit {
  protected logger: Logger = new Logger(SgiClientService.name);

  onModuleInit() {
    // TODO: ถ้า deploy SGI แยก service ให้เพิ่ม API_SGI_BACKEND_* ใน AppConfigService
    // ตอนนี้ใช้ store backend ตัวเดียวกับ StoreClientService
    this.defaultHeaders[this.config.api.store.key.name] = this.config.api.store.key.value;
    this.baseUrl = this.config.api.store.url;
  }
}
// BaseClientService และ { success, data } ให้แล้ว — service ฝั่ง BFF จึงได้ data ตรงๆ
// TODO: เพิ่ม SgiClientService ใน providers(exports) ของ ClientServiceModule (@Global)
// src/modules/sgi-document-list-search/sgi-document-list-search.service.ts (BFF)
import { Injectable } from '@nestjs/common';
import { SgiClientService } from '@common/client-services/sgi-client.service';

@Injectable()
export class SgiDocumentListSearchBffService {
  constructor(private readonly client: SgiClientService) {}

  // BFF ไม่มี DB — หน้าทีเดียวคือแนบ user context แล้ว forward
  private userHeaders(user: any) {
    return {
      'x-user-id': user?.userId,
      'x-user-group-id': user?.groupId,
      'x-user-permissions': (user?.permissions ?? []).join(','),
    };
  };

  getSgiDocumentTasks(params: any, user: any) {
    return this.client.get('/api/v1/sgi/document/tasks', { params, headers: this.userHeaders(user) });
  }

  getSgiDocument(params: any, user: any) {
    return this.client.get('/api/v1/sgi/document', { params, headers: this.userHeaders(user) });
  }
}

// ----- src/modules/sgi-document-list-search/sgi-document-list-search.controller.ts (BFF) -----
import { Body, Controller, Delete, Get, Param, Post, Put, Query, Req, UseGuards } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

// เลือก prefix แบบเดียวทั้งโมดูล: ใช้ '/bff/sgi/...' (ห้ามปนกับแบบไม่มี /bff)
@Controller('bff/sgi/document-list-search')
@UseGuards(AuthGuard('jwt'))
export class SgiDocumentListSearchBffController {
  constructor(private readonly service: SgiDocumentListSearchBffService) {}

  // proxy ของ GET /api/v1/sgi/document/tasks
  @Get('sgi/document/tasks')
  getSgiDocumentTasks(@Query() query: any, @Req() req: any) {
    return this.service.getSgiDocumentTasks(query, req.user);
  }

  // proxy ของ GET /api/v1/sgi/document
  @Get('sgi/document')
  getSgiDocument(@Query() query: any, @Req() req: any) {
    return this.service.getSgiDocument(query, req.user);
  }
}
// TODO: register module ใน app.module.ts ของ BFF และเพิ่ม SgiClientService ใน ClientServiceModule (@Global)

```

10. Database SQL

10.1 ตารางที่อ่าน/เขียน

Table / Object	R/W	Usage
sgi_compensation_documents	R	ค้นเอกสารตาม year/status/store
sgi_impacted_stores	R	ชื่อร้าน ภาค และข้อมูลร้าน
sgi_fgi_impact_sales_summaries	R	flag ข้อมูลผิดปกติ/ยอดขายไม่ครบ 60 วัน
sgi_consideration_logs	R	ผลการพิจารณาสุดท้าย — คัดเอกสารที่จบด้วย หยุดชดเชยประกันรายได้ เข้าคิวของ section 06 (SDD สไลด์ 46 ข้อ 1.9)
workflow_transaction	R	ใช้ของระบบเดิม: workflow engine @srm/glb-workflow
workflow_approver	R	ใช้ของระบบเดิม: workflow engine @srm/glb-workflow

10.2 SQL จริงต่อ Endpoint

GET /api/v1/sgi/document/tasks — Inbox tasks API

```
-- □□ ชื่อคอลัมน์ต่อไปนี้เป็นชื่อ entity ที่หัวข้อ Entity ของเอกสารนี้ประกาศไว้:
-- total_compensation_amount -> compensate_amount
-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
-- □□ ไม่มีตาราง workflow_tasks ของ SGI แล้ว — กล้องงานอ่านจาก engine กลาง (schema sps_store)
-- getPendingFlowByUser({userData})
-- □ DP-1 บิดแล้ว: reference_id = sgi_compensation_documents.id (surrogate · varchar(255)) · □□ DP-2 workflow_transaction ไม่มี PK/index
(19,283 แถว → seq-scan) ห้ามแก้ schema ของ library
-- ห้ามแก้ schema ของ library — กันซ้ำที่ระดับ application ของ SGI
WITH wh AS (
  -- workflow_transaction ไม่มี created_date — ใช้เวลา event แรกจาก workflow_history แทน
  SELECT transaction_id, MIN(create_date) AS first_event_date
  FROM sps_store.workflow_history GROUP BY transaction_id
)
SELECT d.round_no AS "roundNo",
       d.doc_no AS "docNo",
       d.impacted_store_code AS "impactedStoreCode",
       s.store_name AS "impactedStoreName",
       s.zone_cd AS "regionCode",
       GREATEST(COALESCE(-ss.growth_rate_diff, 0), 0) AS "salesDeclinePercent",
       d.total_compensation_amount AS "totalCompensationAmount",
       d.status_code AS "statusCode",
       d.current_section_code AS "currentSection",
       GREATEST(CURRENT_DATE - wh.first_event_date::date, 0) AS "daysPending",
       ss.total_working_days AS "salesDataDays"
FROM sps_store.workflow_approver a
JOIN sps_store.workflow_transaction w ON w.transaction_id = a.transaction_id
JOIN sgi_compensation_documents d ON d.id::text = w.reference_id -- DP-1 = surrogate id -- DP-1
JOIN store s ON s.store_id = d.impacted_store_code
LEFT JOIN sgi_fgi_impact_sales_summaries ss ON ss.impact_process_id = d.impact_process_id
WHERE a.state_id = :sectionFromJwt AND a.state_id = w.current_state_id AND w.version_id = :sgiVersionId
ORDER BY w.update_date
LIMIT :size OFFSET :offset;
```

GET /api/v1/sgi/document — Document search API

```
-- □□ ชื่อคอลัมน์ต่อไปนี้เป็นชื่อ entity ที่หัวข้อ Entity ของเอกสารนี้ประกาศไว้:
-- total_compensation_amount -> compensate_amount
-- d.year -> d.account_year
-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
-- ต้องระบุ :year เสมอ ไม่งั้นตอบ 400 (กติกากำหนดทางธุรกิจ)
SELECT d.round_no AS "roundNo",
       d.doc_no AS "docNo",
       d.impacted_store_code AS "impactedStoreCode",
       s.store_name AS "impactedStoreName",
       s.zone_cd AS "regionCode",
       GREATEST(COALESCE(-ss.growth_rate_diff, 0), 0) AS "salesDeclinePercent",
       d.total_compensation_amount AS "totalCompensationAmount",
       d.status_code AS "statusCode",
       d.current_section_code AS "currentSection",
       -- workflow_transaction ไม่มี created_date (มีแค่ update_date) — วันที่เริ่มงานเอาจาก workflow_history
       CASE WHEN w.current_status_id <> :statusDone THEN GREATEST(CURRENT_DATE - wh.first_event_date::date, 0) ELSE 0 END AS
"daysPending",
```

```

    ss.total_working_days AS "salesDataDays"
FROM sgi_compensation_documents d
JOIN store s ON s.store_id = d.impact_store_code
LEFT JOIN sgi_fgi_impact_sales_summaries ss ON ss.impact_process_id = d.impact_process_id
LEFT JOIN sps_store.workflow_transaction w ON w.reference_id = d.id::text -- DP-1 = surrogate id (reference_id เป็น varchar(255)) AND
w.version_id = :sgiversionid -- DP-2 (ไม่มี index → seq-scan)
WHERE d.year = :year
    AND (:impact_store_code IS NULL OR d.impact_store_code = :impact_store_code)
    AND (:status IS NULL OR d.status_code = :status)
ORDER BY d.doc_no DESC
LIMIT :size OFFSET :offset;

```

10.3 Index / Constraint ที่ควรมี (ข้อเสนอ)

Table	DDL ที่เสนอ	ที่มา / หมายเหตุ
sgi_fgi_impact_sales_summaries	CREATE INDEX idx_sgi_fgi_impact_sales_summaries_impact_process_id ON sgi_fgi_impact_sales_summaries (impact_process_id);	ข้อเสนอ — อนุมานจากคอลัมน์ที่ปรากฏใน WHERE/JOIN ของ SQL ด้านบน ต้องวัด EXPLAIN ก่อนใช้จริง
sgi_compensation_documents	CREATE INDEX idx_sgi_compensation_documents_year_impacted_store_code_status ON sgi_compensation_documents (year, impacted_store_code, status_code);	ข้อเสนอ — อนุมานจากคอลัมน์ที่ปรากฏใน WHERE/JOIN ของ SQL ด้านบน ต้องวัด EXPLAIN ก่อนใช้จริง

ทั้งหมดเป็น ข้อเสนอ ไม่ใช่ข้อกำหนดจาก ข้อกำหนดทางธุรกิจ — ให้ตรวจกับ EXPLAIN ANALYZE บนข้อมูลจริง และรวมเข้าไฟล์ sql/deploy-sgi-*.sql แบบ idempotent (CREATE INDEX IF NOT EXISTS) ตาม pattern ที่ทีมใช้อยู่

11. Processing Flow

Step	Description
1	Read JWT section/role
2	Validate year for documents
3	Build filter query
4	Join sgi_impacted_stores
5	Return page result

12. Acceptance Criteria

- year missing fails for /sgi/document
- leading zero storeCode preserved
- pagination returns total
- status filter works

13. Developer Test Checklist

No	Test
1	tasks by section
2	documents missing year
3	store search
4	empty result

14. Unit Test Scope

6 ชั่วโมง (30% ของ implementation 20 ชั่วโมง) · เครื่องมือ: Jest + mock repository/DataSource (ไม่ต่อ DB จริง)

หัวข้อนี้คือ unit test ที่ต้องเขียนคู่กับโค้ด — ต่างจาก *Developer Test Checklist* ซึ่งเป็น scenario ระดับ end-to-end/manual ที่ใช้ตอนตรวจรับ · รายการด้านล่าง derive จาก field/validation, acceptance criteria, endpoint และตารางที่เอกสารนี้เขียน

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
docNo	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required when opening existing document · รูปแบบ: YYYY/xxxx
storeCode	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: numeric length = 5 · รูปแบบ: string 5 digits
amount	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: ≥ 0 · รูปแบบ: number, 2 decimals
percent	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: 0-100 · รูปแบบ: number, 2 decimals
sourceSystem	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: ALLMAP / USER · รูปแบบ: enum
date	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: valid date · รูปแบบ: DD/MM/YYYY
attachment	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: ≤ 5 MB · รูปแบบ: file
year	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required for /sgi/document · รูปแบบ: ค.ศ. YYYY
page/size	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: $\text{page} \geq 1$ $\text{size} \leq 100$ · รูปแบบ: integer
business rule	logic	year missing fails for /sgi/document
business rule	logic	leading zero storeCode preserved
business rule	logic	pagination returns total
business rule	logic	status filter works
GET /api/v1/sgi/document/tasks	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
GET /api/v1/sgi/document	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
service	error mapping	แปลง error ของ repository/lib เป็น error code ตามสัญญากลาง (LLDD-BE-API-Common-Contracts.pdf)

- ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง — mock ที่ขอบ repository/client เสมอ
- ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่
- เกณฑ์ผ่านของ CI: ทุกเคสในตารางนี้มี test จริงและผ่านทั้งหมด